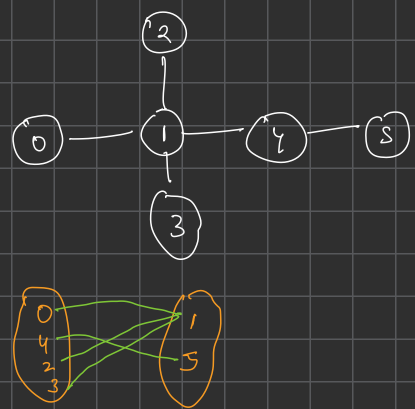
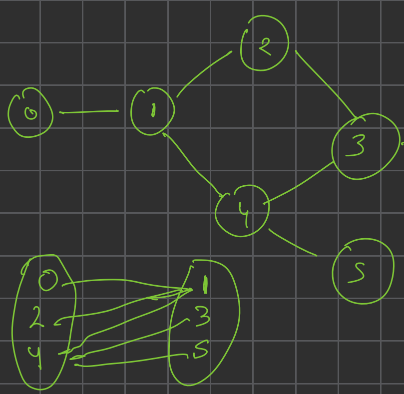
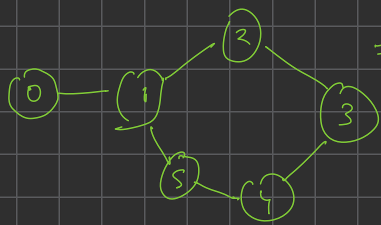
[Bipartite graph]:

Node ∃ 2 group distribute,
Node in a same should not be connected through a edge

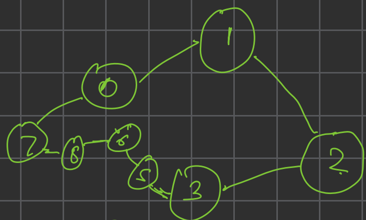


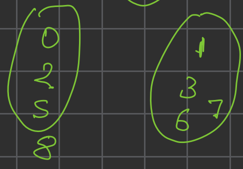
Bipartite graph

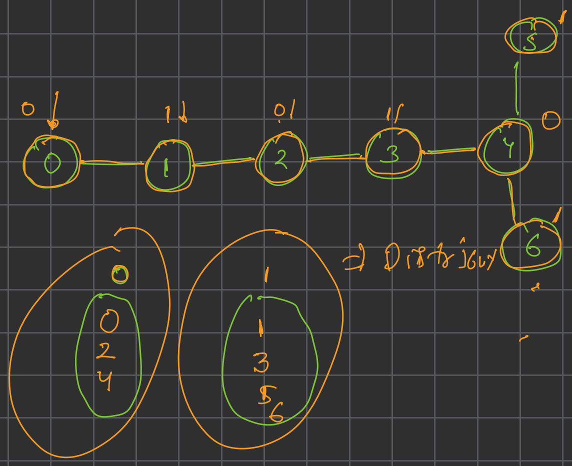




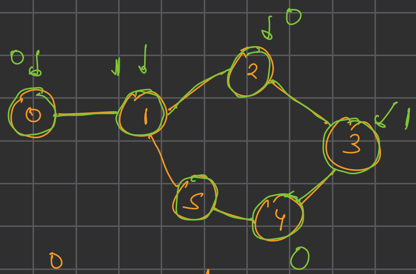
⇒ odd length
cycle present
⇓
Bipartite
exist







0 1 2 3 4 5 6
0
0 2 4
1
1 3 5 6



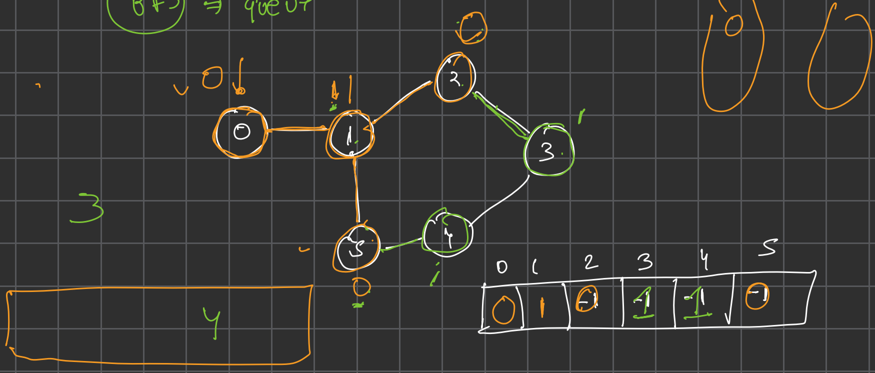
0

0
2
4

1

1
3

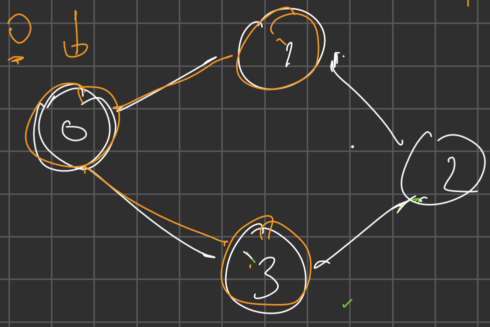
BFS ⇒ queue



$$Color[1] = (color[0]+1)\%2$$

$$(0+1)\%2$$

$$(1+1)\%2$$



0	1	2	3
0	1	-1	1

color

1	3

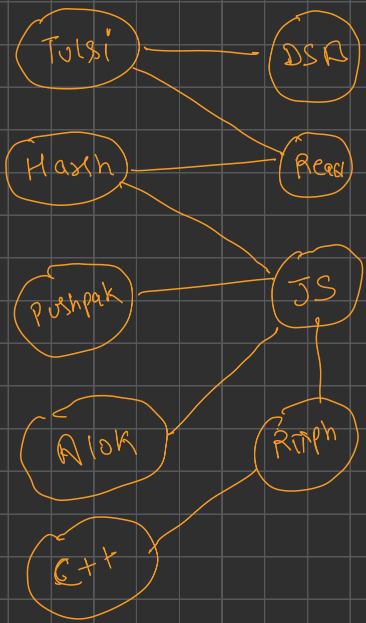
return 1

0

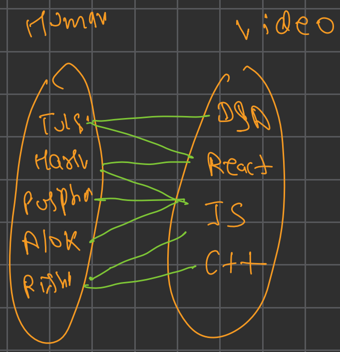
Bipartite

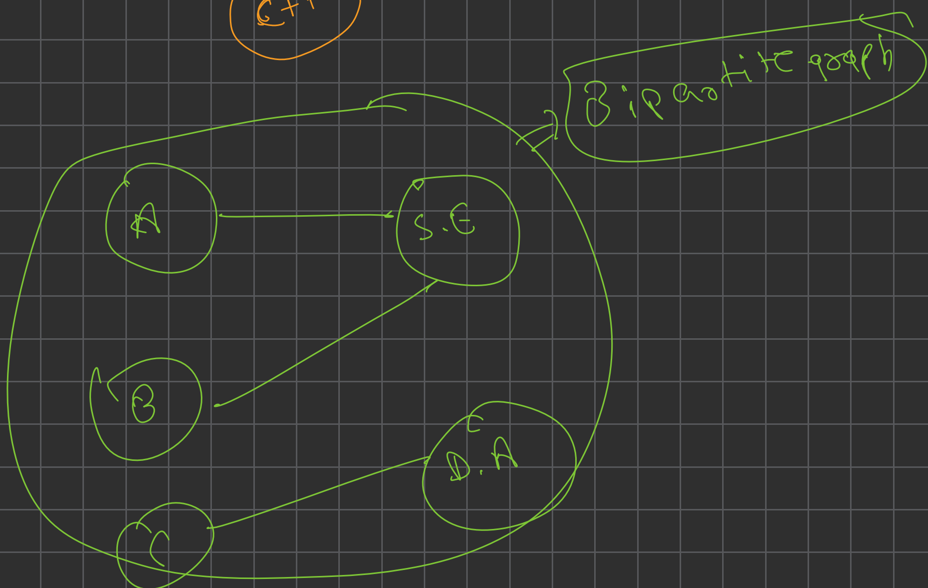
1. Create a color vector of size V and assign all values -1
2. push 0 into queue and assig color as 0 to it
3. While queue is not empty
   1. Get the first node
   2. Look at its all neighbours
      - → if neigb color is -1 (pados color assig)
        color[neib] = (color[node]+1)%2;
        push neib into queue
      - → if neibor color == node
        return 0;

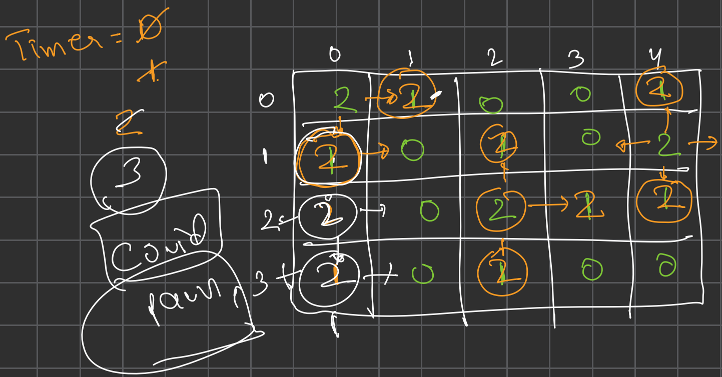
1



Youtube recommend





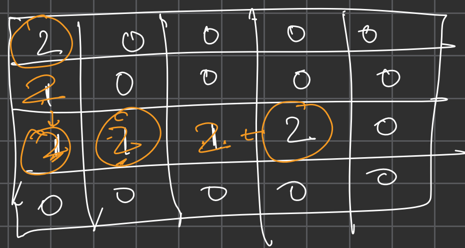


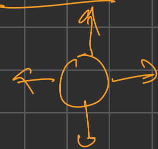
0 - Empty
1 = Unaffected parts
2 - covid parts

	0	1	2	3	4
0	2	1	0	0	1
1	1	0	1	0	2
2	1	0	2	1	1
3	1	0	1	0	0

2 second

1st second

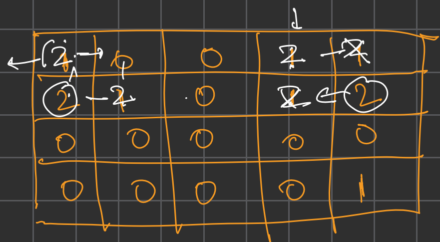




2 second

1 second

2 second



	0	1	2	3	4
0	0	2	2	0	0
1	2	2	0	0	0
2	2	2	0	2	2
3	2	2	0	2	0

Timer = ~~0~~

Timer = 1
~~2~~
~~3~~
4

4 - 1 => 3 answer

hospital[-1][r-1]
→ error

	0	1	2	3
0	2	0	2	0
1	2	2	2	0
2	0	1	2	0
3	0	2	1	0

Timer = ~~0~~, ~~1~~, $\boxed{2}$ ✓

Current patient = 4

(1,0) (1,1) (0,2) (2,2)

(i,j)

(0,0)

(0,-1) (0,1) (1,0) (-1,0)

(i,j-1) (i,j+1) (i+1,j) (i-1,j)

2>=0

2<5

2>=0

2<3

=> Out of index

2, 2

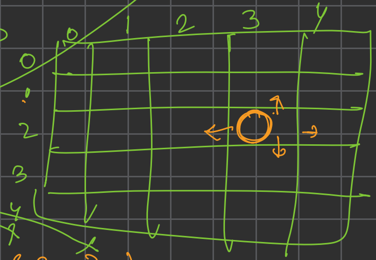
2+0 3+-1

i j

(i, j) => (2, 3)

(2, 2) (2, 4) (1, 3) (3, 3)

Left Right Up down



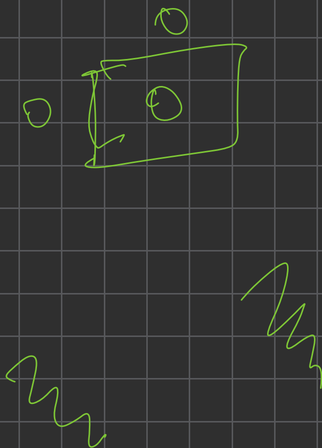
int row[4] = {0, 0, -1, 1}
int col[4] = {-1, 1, 0, 0}

Looop

```
for(int k=0; k<4; k++)
{
    if(hospital[i+row[k]][j+col[k]])

}
```

int row[4] = {0, 0, 1, -1}
int col[4] = {1, -1, 0, 0}



Timer = 0

return Timer - 1

-1

Pillars of OOPS

Abstrac

MC

↓

AC

↓

PL / FL

↓

$\boxed{\text{OOPS}}$

Class → Object

Abstraction

$\boxed{\text{Constructors}}$

Class → object

Pillars of OOPS

- Abstraction
- Encapsulation
- Inheritance
- polymorphism.

```
class Student {
    string name;
    int age;
    int rN;
    String College;
}
```

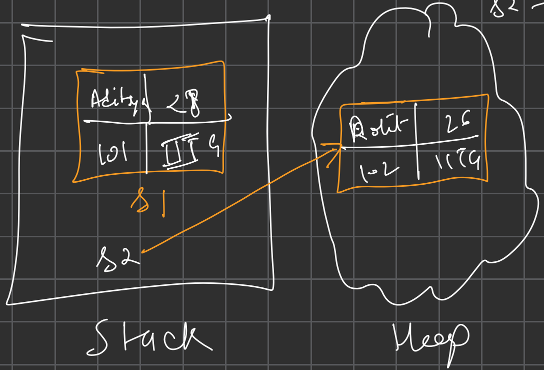
```
Student s1;

s1.name = "Aditya"

int x

Student *s2 = new Student();

s2 -> name = "Rohit"
s2 -> age = 26.
```




```
void f1() {
    int x;
    string y;
    Student *s1 =
        new Student();
}
```

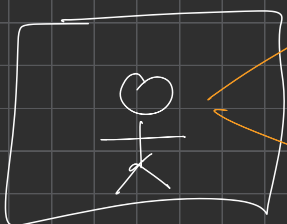
OOPS

Pillars

Encapsulation

↳ En + Capsule

1



Student

→ Characters
(name, age, ---)

→ Behaviours
(markAttendance,
giveExam, --)

2 Data Security

BankAccount

↳ atm pin

↳ amount

①

```
Class Student {
  Public:
    String name;
    int age;

    void markAttendance() {

    }
};
```

variables

Methods

② Data Security

Access Modifiers

- Private
- Protected
- Public